# Python Code

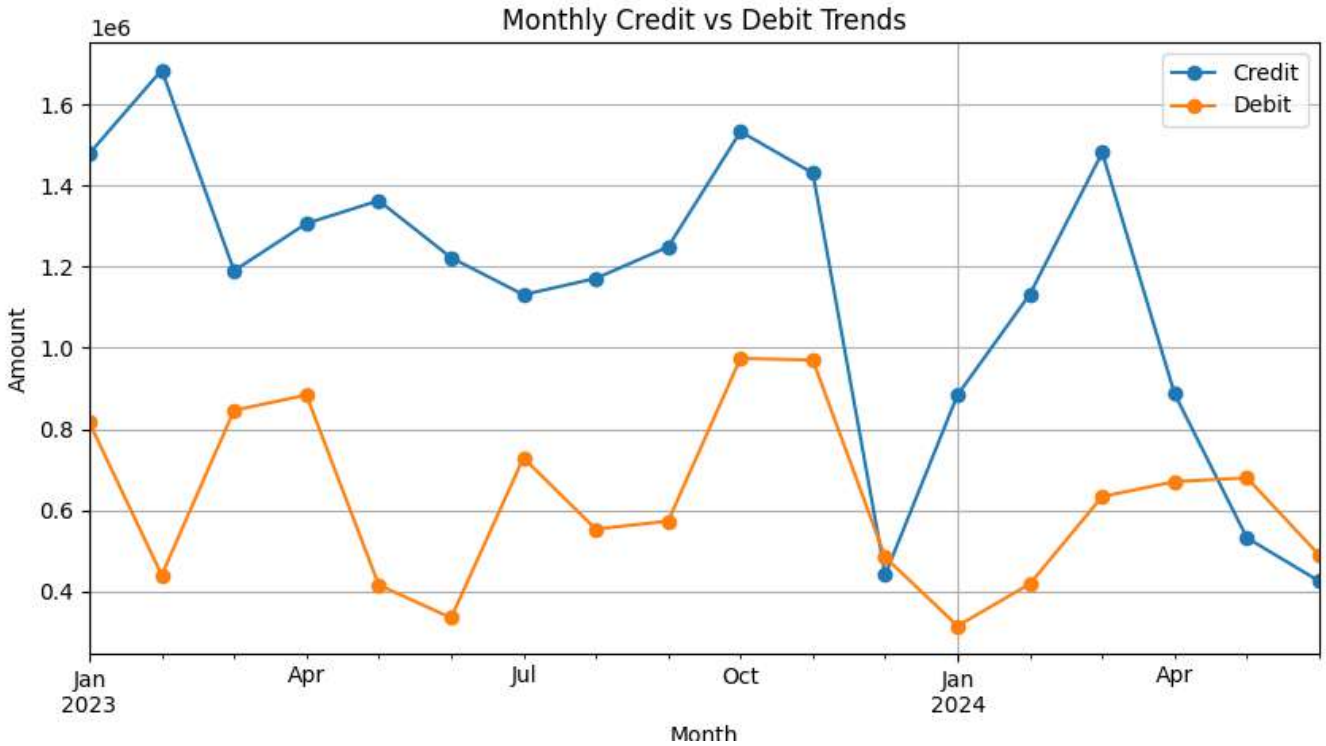
```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats



# Task 1

df = pd.read_csv(r"D:\Data Science\Python\New folder\jp_morgan.csv")

df['TransactionDate'] = pd.to_datetime(df['TransactionDate'], dayfirst=True)

cols_to_fix = ['TransactionAmount', 'AccountBalance']
for col in cols_to_fix:
    if df[col].dtype == 'object':
        df[col] = df[col].str.replace(r'[^\d.]', '', regex=True).astype(float)

df['AccountType'] = df['AccountType'].str.capitalize()
df['TransactionType'] = df['TransactionType'].str.capitalize()

df.ffill(inplace=True)

print("Data Types after cleaning:\n", df.dtypes)
print("\nFirst 5 rows:\n", df.head())

# Task-2 ////////////////////////////////////////

# Better & faster approach
df['Credit'] = df['TransactionAmount'].where(df['TransactionType'].str.lower().isin(['deposit', 'credit']), 0)
df['Debit'] = df['TransactionAmount'].where(df['TransactionType'].str.lower().isin(['withdrawal', 'payment', 'transfer', 'debit']), 0)


df['Year'] = df['TransactionDate'].dt.year
df['Month'] = df['TransactionDate'].dt.to_period('M')

monthly_summary = df.groupby('Month')[['Credit', 'Debit']].sum()
yearly_summary = df.groupby('Year')[['Credit', 'Debit']].sum()

print(monthly_summary.head())
print(yearly_summary)

monthly_summary.plot(kind='line', figsize=(10,5))
plt.title("Monthly Credit vs Debit Trend")
plt.xlabel("Month")
```

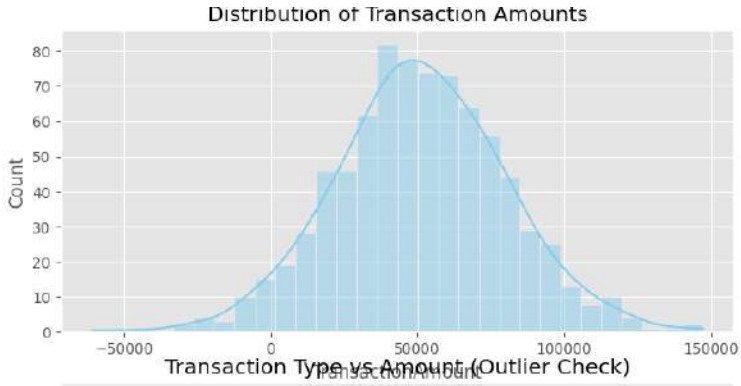
```python
plt.ylabel("Amount")
plt.show()


account_performance = df.groupby('AccountID')[['Credit','Debit']].sum()
account_performance['Net_Inflow'] = account_performance['Credit'] - account_performance['Debit']

top_accounts = account_performance.nlargest(5, 'Net_Inflow')
bottom_accounts = account_performance.nsmallest(5, 'Net_Inflow')

print("\nTop 5 Accounts:\n", top_accounts)
print("\nBottom 5 Accounts:\n", bottom_accounts)

df = df.sort_values(by=['AccountID', 'TransactionDate'])
df['Days_Since_Last_TX'] = df.groupby('AccountID')['TransactionDate'].diff().dt.days

dormant_accounts = df[df['Days_Since_Last_TX'] >= 60]['AccountID'].unique()

df['Status'] = df['AccountID'].isin(dormant_accounts).map({True: 'Dormant', False: 'Active'})

print(f"\nTotal Dormant Accounts: {len(dormant_accounts)}")
print("Sample Dormant Accounts:", dormant_accounts[:5])

# --- TASK 3: CUSTOMER PROFILE BUILDING ---

# 1. Rubric for Activity Levels (Heading required as per JD)
# High Activity: > 10 transactions
# Medium Activity: 5 to 10 transactions
# Low Activity: < 5 transactions

# Transaction frequency calculate karna
tx_frequency = df.groupby('AccountID').size().reset_index(name='Tx_Count')

def classify_activity(count):
    if count > 10: return 'High'
    elif count >= 5: return 'Medium'
    else: return 'Low'

tx_frequency['ActivityLevel'] = tx_frequency['Tx_Count'].apply(classify_activity)

# 2. Segmenting by Average Balance and Transaction Volume
customer_profiles = df.groupby('AccountID').agg({
    'AccountBalance': 'mean',
    'TransactionID': 'count'
}).rename(columns={'AccountBalance': 'Avg_Balance', 'TransactionID': 'Total_Transactions'})

# 3. Specific Profile Identification

# Profile A: High-net inflow accounts (Top 25% of net inflow from Task 2)
high_net_inflow_threshold = account_performance['Net_Inflow'].quantile(0.75)
```

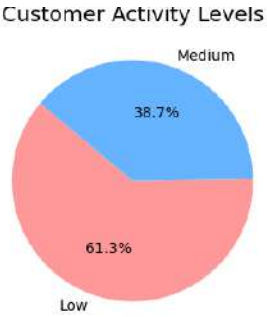
```python
high_net_accounts = account_performance[account_performance['Net_Inflow'] >= high_net_inflow_threshold]

# Profile B: High-frequency low-balance accounts
# (High activity but Average Balance in bottom 25%)
low_bal_threshold = customer_profiles['Avg_Balance'].quantile(0.25)
high_freq_low_bal = customer_profiles[
    (customer_profiles['Total_Transactions'] > 10) &
    (customer_profiles['Avg_Balance'] < low_bal_threshold)
]

# Profile C: Accounts with negative or near-zero balances (Threshold < 500)
near_zero_balance_accounts = customer_profiles[customer_profiles['Avg_Balance'] < 500]

# --- RESULTS ---
print("--- Activity Level Distribution (Rubric: High>10, Med 5-10, Low<5) ---")
print(tx_frequency['ActivityLevel'].value_counts())

print("\n--- Profile Counts ---")
print(f"High-Net Inflow Accounts: {len(high_net_accounts)}")
print(f"High-Frequency Low-Balance: {len(high_freq_low_bal)}")
print(f"Near-Zero Balance Accounts: {len(near_zero_balance_accounts)}")



# --- TASK 4: FINANCIAL RISK IDENTIFICATION ---

# 1. Track Overdrafts (Balance <= 0)
overdraft_accounts = df[df['AccountBalance'] <= 0]['AccountID'].unique()

# 2. Large Withdrawals (Top 10% of all withdrawals)
withdrawal_threshold = df[df['TransactionType'] == 'Withdrawal']['TransactionAmount'].quantile(0.90)
large_withdrawals = df[(df['TransactionType'] == 'Withdrawal') & (df['TransactionAmount'] > withdrawal_threshold)]

# 3. Balance Volatility (Standard Deviation of Account Balance per Account)
volatility = df.groupby('AccountID')['AccountBalance'].std().reset_index(name='Balance_Volatility')

# 4. Outlier Detection using IQR (for TransactionAmount)
Q1 = df['TransactionAmount'].quantile(0.25)
Q3 = df['TransactionAmount'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

anomalies = df[(df['TransactionAmount'] < lower_bound) | (df['TransactionAmount'] > upper_bound)]

# 5. Risk Flagging
# Aisi customers jinme high volatility hai aur anomalies bhi hain
high_vol_threshold = volatility['Balance_Volatility'].quantile(0.75)
```

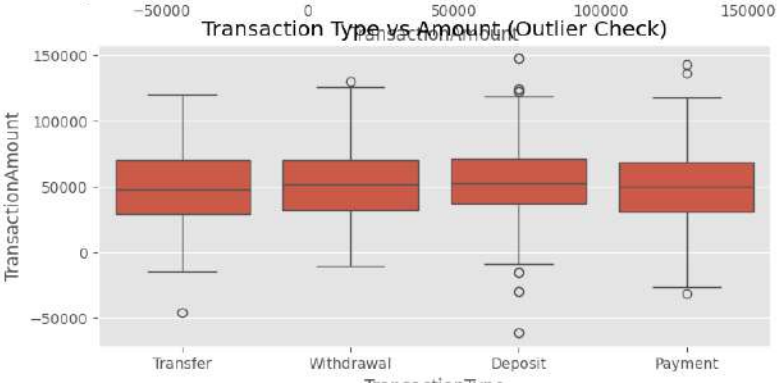
```python
risky_accounts = volatility[volatility['Balance_Volatility'] > high_vol_threshold]['AccountID'].tolist()

# Results Print Karein
print(f"--- Risk Analysis Summary ---")
print(f"Accounts with Overdrafts: {len(overdraft_accounts)}")
print(f"Number of Large Withdrawals (> {withdrawal_threshold:.2f}): {len(large_withdrawals)}")
print(f"Number of Transaction Anomalies detected: {len(anomalies)}")

print("\n--- Top 5 Most Volatile Accounts (High Risk) ---")
print(volatility.sort_values(by='Balance_Volatility', ascending=False).head(5))


# Task-5/////////////
```

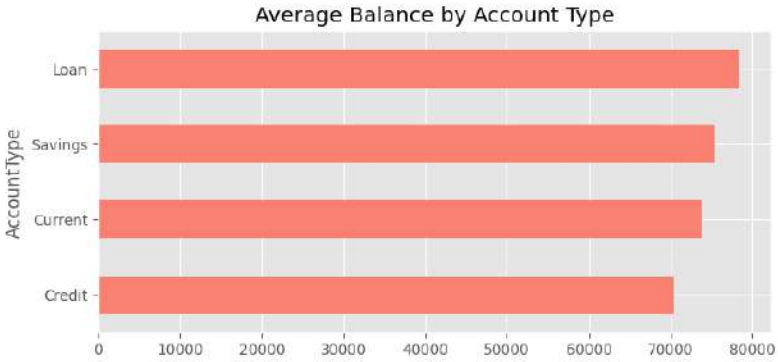
```python

high_volume_ids = tx_frequency[tx_frequency['Tx_Count'] > median_tx]['AccountID']
low_volume_ids = tx_frequency[tx_frequency['Tx_Count'] <= median_tx]['AccountID']

# 2. find average
high_vol_balances = customer_profiles.loc[high_volume_ids, 'Avg_Balance']
low_vol_balances = customer_profiles.loc[low_volume_ids, 'Avg_Balance']

# 3. T-Test
t_stat, p_value = stats.ttest_ind(high_vol_balances, low_vol_balances, equal_var=False)

# 4. Print Result
print(f"--- Hypothesis Testing Results ---")
print(f"Median Transaction Count: {median_tx}")
print(f"Average Balance (High Volume): {high_vol_balances.mean():.2f}")
print(f"Average Balance (Low Volume): {low_vol_balances.mean():.2f}")
print(f"P-Value: {p_value:.4f}")

# 5. Interpretation
alpha = 0.05 # 5% Significance level
if p_value < alpha:
    print("\nResult: SIGNIFICANT. Reject Null Hypothesis.")
else:
    print("\nResult: NOT SIGNIFICANT. Not reject Null Hypothesis.")

```


Monthly Credit vs Debit Trends
1e6
1.6
1.4
1.2
1.0
0.8
0.6
0.4
Amount
Credit
Debit
Jan
2023
Apr
Jul
Oct
Jan
2024
Apr
Month


Distribution of Transaction Amounts
Count
80
70
60
50
40
30
20
10
0
−50000
0
50000
100000
150000


Customer Activity Levels
Medium
38.7%
61.3%
Low


Transaction Type vs Amount (Outlier Check)
TransactionAmount
150000
100000
50000
0
−50000
Transfer
Withdrawal
Deposit
Payment


Average Balance by Account Type
AccountType
Loan
Savings
Current
Credit
0
10000
20000
30000
40000
50000
60000
70000
80000